ML4NLP 1

Tutorial 14: Exam preparation

15. December 2025

Schedule & Admin

- This week
  - Going over the Exam Format
  - Further Involvement with CL / NLP
  - Exam preparation through sample exam questions

- Peer Review for Ex.6 starts today (deadline: 22.12.2025)

General Exam Style

- A4 cheat sheet allowed (both sides handwritten, one side if printed)
- Exam questions: Mostly Conceptual, basic calculations or code snippets can come up
- Primarily open ended questions on the Lecture Content, some True/False and multiple choice (up to 40% of the exam points)
- No negative grading
  - It’s always better to answer something (don’t go off topic, make short and precise guesses)
- Elaborate use case scenarios questions are to be expected
  - (E.G) You are an NLP Engineer and you are asked to design system X using the following material Y.
- Conceptual questions about code discussed in tutorials might come up…

Further Involvement with CL / NLP

Interesting classes (that are also applicable for Informatics, CL students know):

Fall Semester:

- Advanced Techniques of Machine Translation
- Machine Learning for Natural Language Processing 1

Spring semester:

- Advanced Machine Learning
- Machine Learning for Natural Language Processing 2 (Simon, Andrianos)
- Retrieval Augmented Generation

Open Master Thesis topics within Computational Linguistics / NLP (e.g under supervision of Michelle or Andrianos or Simon)

https://www.cl.uzh.ch/en/studies/studies-BA-MA/teaching/BaMasterArbeit.html

Exam: Common Topics

- Historical development of NLP
- High-level questions about challenges in NLP
- Use case scenarios in Industry settings
- General ML concepts
- Word embeddings
- Clustering
- Multitask Learning
- Transfer learning
- Computation Graphs
- Neural Network model types:
  - FFNNs
  - RNNs
  - CNNs
  - Transformers
- Pre-trained LMs
- Contextualized Embeddings
- Pre-trained / Chat-tuned LLMs

Examples of High-level Questions

1. What makes ML for NLP challenging? Mention two important and general sources/types of difficulties of NLP. For each type, mention one current technique that helps to alleviate it and why it roughly works.

2. What are two instances of unsupervised learning techniques used in NLP.

General ML

Joe claims that for a complex machine learning task, where various features need to be considered, the following is true: training a single **linear regression model** with 500 features is equivalent to building an ensemble of 500 models each with a single feature.

Would you **agree**, yes or no? Add a **short justification** for your answer.

No, I would not agree with Joe's claim. This assertion overlooks key aspects of model complexity and feature interaction.

- Linear regression with multiple features can capture interactions among variables, which is essential in understanding complex relationships in data.

- An ensemble of single-feature models lacks this capacity, as each model is isolated and unable to account for how features might interact with one another.

This fundamental difference in how these models handle feature interactions leads to distinct outcomes in their predictive capabilities and accuracy.

Word Embeddings

1. What are the commonalities / differences between the following?
   - word2vec
   - Glove
   - ELMo
   - BERT
2. Name two advantages of contextualized word embeddings over static word embeddings

2. Can incorporate syntactic information, don't mix all meaning of a word in one average representation (e.g., ambiguous words)

1. Commonalities:
   - Predictive method to learn word representations given distributional semantics (context)

Differences:
   - word2vec uses sliding windows running over all training samples
   - GloVe learns from the global co-occurrence matrix directly to leverage statistical information in the entire corpus
   - ELMo uses a bidirectional LSTM trained to predict the next word in the sequence (left-to-right and right-to-left) and combines the hidden representations from both LSTM to get contextualized embeddings
   - BERT uses a transformer encoder-only architecture trained with MLM also for contextualized embeddings

GPT

1. How would you explain to a layperson the important ideas behind GPT3 in your own words?

    - 175B parameters and was trained on web-crawled text, books and code
    - pretrained using generative language modeling - learns to predict the next word in a piece of text
    - words are not “words” per se, but “subtokens” - allows GPT to represent and produce also rare words with a limited vocabulary
    - It’s architecture is based on a transformer decoder that process a fixed amount of subtokens in a single forward pass.
    - A GPT-like model *can* be applied to any text-based task by framing it as a prompt that elicits the correct answer as a valid continuation.

Language Modelling Objectives

1. What's the difference between the following learning objectives?
    - Masked Language Modelling
    - Generative Language Modelling

Under which setting are these two learning objectives identical?

Masked Language Modelling is a form of cloze (fill in the gap) pre-training learning objective where the model has to learn to predict the missing words (15-20%) (<MASK>) given its surrounding context.

Generative Language Modelling is an "autocomplete" pre-training learning objective where the model learns to autoregressive predict the next token as a native task to text generation

Masked Language Modeling becomes equivalent to Generative (autoregressive) Language Modeling when you mask exactly the final token of each sequence and enforce a causal attention mask, so the model can see only past tokens.

Language Modelling Objectives Multiple Choice

The XLM model called xlm-mlm-tlm-xnli15-1024 is pretrained with which of the following tasks:

- ☑ Masked Language Modelling
- ☐ Generative Language Modelling
- ☑ Translational Language Modelling
- ☐ Next Sentence Prediction

GPT and ELMO have in common that they use generative language modeling for pretraining.

- ☑ True
- ☐ False

GPT and BERT have in common that they use generative language modeling for pretraining.

- ☐ True
- ☑ False

Transformer Architecture

3.b) Attention between every token repr. of the input sequence and every token repr. in the output sequence so far (K/V=Encoder hidden states of the full input sequence; Q=Decoder hidden states of the output sequence so far); located in the decoder

1. Locate the encoder and the decoder in the figure:

2. Label the parts of the architecture marked with ?

   ? = positional encodings

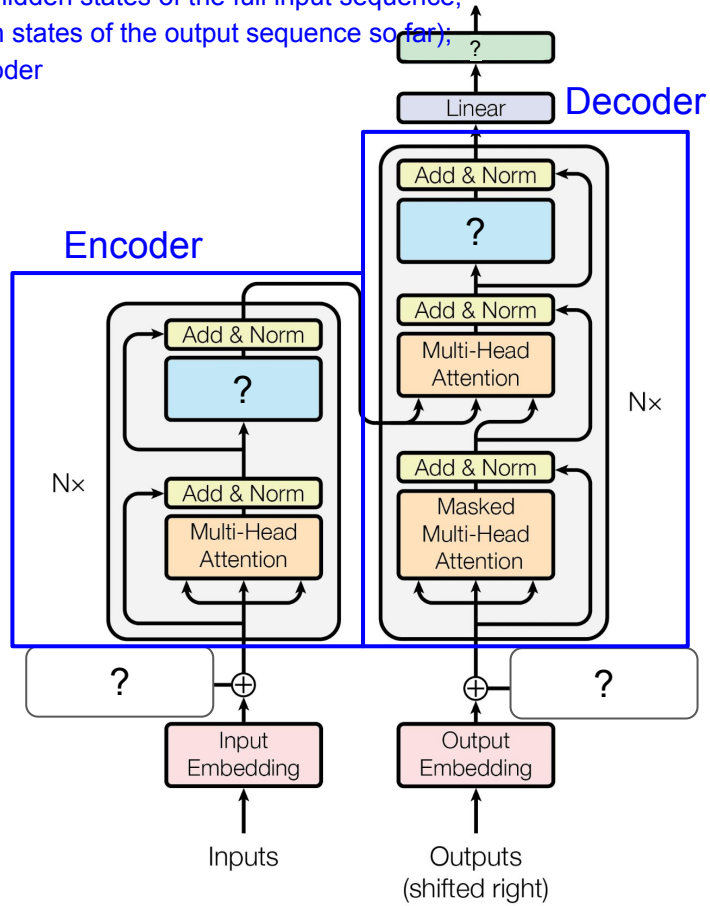
3. What are the differences between the following:

   a. Self-attention in the encoder

   b. Cross-attention

   c. Self-attention in the decoder

a) Attention between every token repr. in the input sequence (Q/K/V=Encoder hidden states of the full input sequence)



c) Attention over the output sequence, but masked so a token cannot attend to future tokens (causal attention; Q/K/V=Decoder hidden states of the output tokens so far)

Pretrained LMs

1. Given the following parameter dictionary for a mini LM, what is the total parameter count of the model? (keys indicate the named params, values indicate their sizes)

```
m = {
    'wpe': [20, 5],
    'wte': [50, 5],
    'ln_f': {'b': [5], 'g': [5]},
    'blocks': [{
        'ln_1': {'b': [5], 'g': [5]},
        'attn': {
            'c_attn': {'b': [20], 'w': [5, 20]},
            'c_proj': {'b': [5], 'w': [5, 5]}
        },
        'ln_2': {'b': [5], 'g': [5]},
        'mlp': {
            'c_fc': {'b': [30], 'w': [5, 30]},
            'c_proj': {'b': [5], 'w': [30, 5]}}
     }],
}
```

Approach:
Parameters are weights (matrices) + biases (vectors, if they exist)
There are N·M parameters in an NxM matrix, N parameters in a vector of dimension N.

Total parameter count:
Count parameters of:

- **Token and positional embeddings**
- **Layer norm**
- **Block = attention, layer norms, MLP**

Sum the above.

Quantization

You are evaluating your company's new internal LLM and you notice that it occasionally generates some unexpected "noisy" tokens out of context. Your team's AI star expert (according to LinkedIn) comes and tells you:

"Just quantize the weights of the model to 4bit, there won't be any noise coming from the model after that"

Is this hypothesis sensical? Why / Why not?

Quantization of the weights of the models results into reduced precision of the model. This might lead to slightly different behaviors but often the model performs similarly.

However, the lost precision from the removal of the "trailing numbers" of the weights of the models is not removing any specific behavior of the model and even less likely, the noisy behaviors. Therefore NO, this is not a sensical hypothesis, it's far from likely to be the case.

LLMs

1. What is the main difference between GPT-3 (the base model described in Brown et al., 2020) and GPT-3.5-Turbo (the model that was presumably powering the first instances of ChatGPT)?

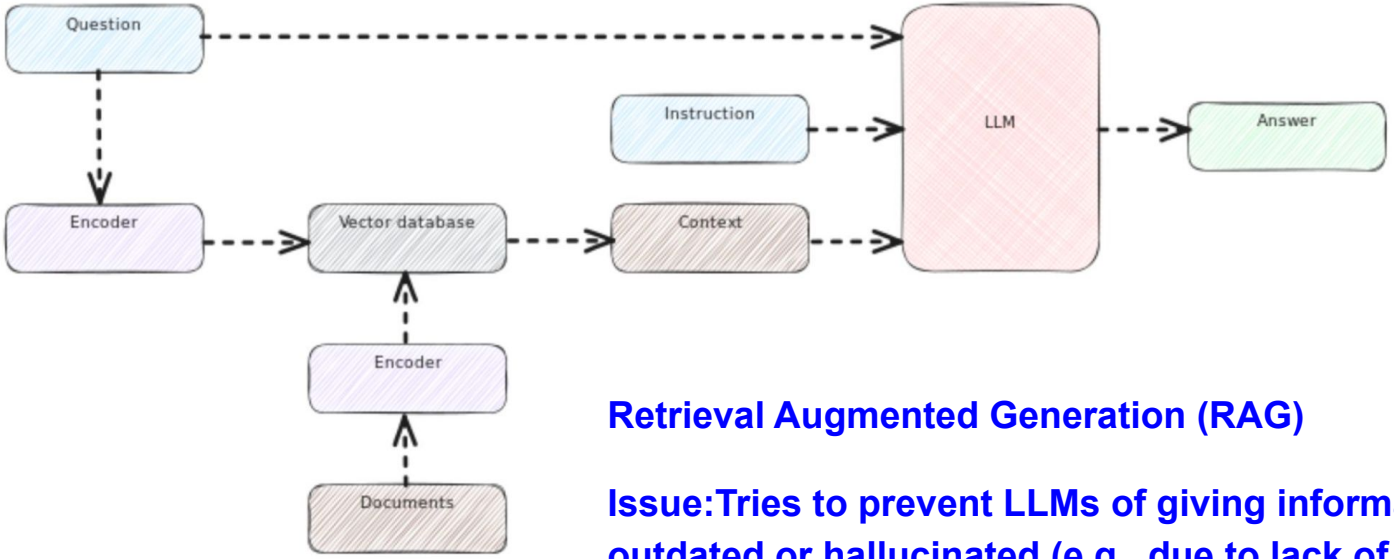
   GPT-3 is purely a generative language model whilst GPT-3.5-Turbo is an instruction tuned model to be able to be used as a chat agent.

2. The management has decided to get on the Generative AI bandwagon and want LLMs integrated into their product offering. However, they are concerned with data privacy and will not allow the use of paywalled API models. What would you suggest as a way forward? What are the pros and cons?

   Privately host open weight models in a setup where privacy can be fully preserved. Downside is that the open weight models are typically worse than the (probably larger) closed source ones and the infrastructure needed to host the model might require some effort and cost to set up and maintain.

LLM Inference

1. What type of technique does the following image illustrate? What issue does it try to fix?



Retrieval Augmented Generation (RAG)

Issue:Tries to prevent LLMs of giving information that is outdated or hallucinated (e.g., due to lack of knowledge on the specific topic, or vague input) --> adds context to prompt and instruction by querying database

Automatic Evaluation Through LLMs

You work in a company where you call a pipeline of LLM based evaluators (LLM as a Judge, no previous results are being stored)) to numerically evaluate (1-5) aspects (e.g eye catching) of texts written by editors before publishing these texts. The format of this prompt for each aspect is:

Instruction: {Length explanation of the task}

Text: {text}
Aspect: *INSERT PREDICTION (1-5)*

John, an editor in the company is wondering whether if he runs his identical text again through the evaluation pipeline could result in improved scores.

Explain to John whether his hypothesis is valid. If it is / is not, what aspect of the evaluation pipeline leads to this behavior/perk?

Next token predictions, when sampling is enabled, remain stochastic, even if temperature is low. There are many factors that could lead to an alternative prediction and therefore it's very likely that his text receives different scores each time it goes through the evaluation pipeline, perhaps higher or lower. You can minimize this effect by setting an extremely low temperature or running evaluation multiple times and taking the average evaluation, but eliminating the variance completely in these predictions won't be possible.

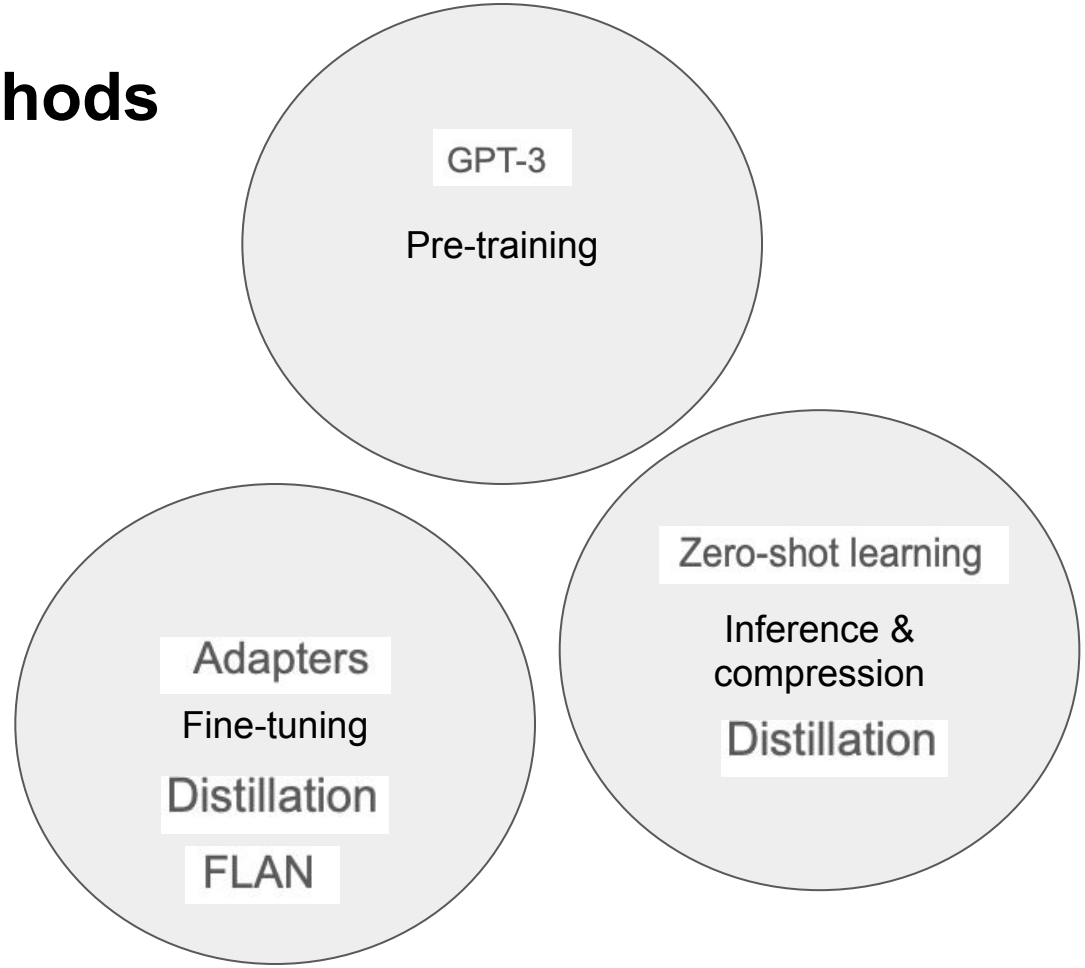
Parameter-efficient methods

Assign the following models/techniques to their umbrella term(s) (can be more than 1).

- GPT-3
- Zero-shot learning
- Distillation
- FLAN
- Adapters

Recommended reading:

[Treviso et al. 2022: Efficient Methods for Natural Language Processing: A Survey]



Parameter-efficient methods

1. Name one advantage and one disadvantage of adapters.

   - Advantage: need to adapt fewer parameters than a full model fine-tuning;

   - Disadvantage: Increased inference time due to more parameters (-> techniques to mitigate this)

2. What's the difference between prompt-tuning and prefix-tuning? And which of the two is more parameter-efficient?

Prompt-tuning:

- no changes are required in the pretrained model weights (parameters)
- Soft prompts differ from the discrete text prompts in that they are acquired through back-propagation and is thus adjusted based on loss feedback from a labeled dataset.

Prefix tuning

- adds trainable tensors to each transformer block instead of only the input embeddings, as in soft prompt tuning.
- Also, we obtain the soft prompt embedding via fully connected layers. These fully connected layers embed the soft prompt in a feature space with the same dimensionality as the transformer-block input to ensure compatibility for concatenation.

Clustering

1. What is the difference between clustering and classification?

   Clustering is an unsupervised learning task, where data points are grouped into clusters without predefined labels (discovering underlying structures/patterns in the data), while **classification** is a supervised learning task where a model learns from labeled data to assign predefined labels to unseen data points (prediction of correct class).

2. What concrete evaluation measurements can be used to assess the quality of topic modeling?
   - **Topic coherence and topic diversity** (Are the top words of a topic similar? Do they co-occur?): Normalized Pointwise Mutual Information (NMI), word embedding similarity or inverse rank-biased overlap (RBO)
   - **Topic exclusivity** (How specific are top words for a topic?): Word probability per topic vs. topic specificity; Mallet tool exclusivity metric (which top words of a topic do not appear as top words in another topic)
   - **Saliency** (How prominent are the top words of a topic within the entire corpus?): distinctiveness x term's frequency
   - **Relevance** (How relevant is a term to a topic?)
   - (**Interpretation**: Conceptual topic names from word distributions; LLM based interpretation; human interpretation)